Assignment: Build a 3-Agent "Learning-Bite" Crew with One Built-in and One Custom Tool

Assignment Brief | Domain: Learning and Development (Kaveri Infotech, a fictional company) | Reference code: github.com/SetuAI/Oracle_26_30_Oct (folder crewAI/: 1_first_agent.py, stock_tool.py, portfolio_tools.py, 4_stock_crew.py)

Level	Time	Stack	You hand in
Beginner to Intermediate	4 - 6 hours	Python 3.10 - 3.13, CrewAI, crewai-tools (ScrapeWebsiteTool), OpenAI gpt-4o-mini, python-dotenv	3 Python files + 2 output posts + notes.md + screenshots

The goal in one line. You will build a crew of three agents that reads a real web page, plans a short lesson from it, and writes a LinkedIn post that is checked by code you wrote. Agent 1 uses CrewAI's built-in **ScrapeWebsiteTool**; Agent 3 uses your own **custom tool**, which measures the post and returns PASS or FAIL. In class you built the Portfolio X-Ray crew with two custom tools. This assignment combines a built-in tool, a custom tool and a write-check-revise loop.

1. The scenario

The Learning and Development (L&D;) team at Kaveri Infotech posts a weekly **learning bite** on LinkedIn: one short post that explains one tech topic (RAG, vector databases, Docker) to freshers. Writing each post by hand takes two hours, so the team tried asking ChatGPT. Two problems appeared every week:

- **Invented facts.** The model wrote from memory, and some facts were out of date or wrong. The team wants every fact to come from one named source page, so it can be checked.
- **Wrong lengths.** The model said "here is your 200-word post" and the post had 340 words. Models cannot count words reliably. LinkedIn posts for this series must be 150 - 250 words, easy to read, with 3 - 5 hashtags.

Your crew fixes both: facts come only from a scraped page, and the length and readability are measured by Python code that the writer agent must call and obey.

2. What happens when you run the crew

#	Agent	Tool	Task: what goes in	What comes out
1	Web Researcher	ScrapeWebsiteTool (built-in)	The URL and topic from crew.kickoff(inputs=...)	A one-line definition + exactly 8 facts, each tagged with its page section
2	Learning Designer	none	Task 1 output (via context)	An outline: hook, definition, 3 key points, example, misconception, call to action
3	LinkedIn Writer	PostQualityCheckerTool (custom)	Task 2 outline + Task 1 facts (via context)	Draft -> checker -> fix -> checker ... until VERDICT: PASS; saved to output/learning_bite.md

The tasks run in order (Process.sequential). Task 2 only sees what Task 1 produced, and Task 3 only sees Tasks 1 and 2, because you pass them through `context=[...]`. If Task 1 extracts wrong facts, Tasks 2 and 3 repeat them, so you test the scrape tool on its own first (Build Guide, Step 1). If the checker says FAIL, the writer revises and calls it again; `max_iter` stops the loop if it never passes.

3. Built-in tool versus custom tool

	ScrapeWebsiteTool (built-in)	PostQualityCheckerTool (custom, you write it)
Comes from	pip install crewai[tools]; import from crewai_tools	Your file post_tools.py, a class that inherits crewai.tools.BaseTool
Input	website_url (string)	post_text (string), declared in a Pydantic args_schema class
Does	Downloads the page and returns its visible text	Counts words, sentences, average sentence length, Flesch Reading Ease and hashtags; compares with 4 rules
Returns	"The following text is scraped website content: ..."	The numbers, then VERDICT: PASS, or VERDICT: FAIL with one fix per broken rule
Why a tool	The model cannot open web pages by itself	The model cannot count words or compute a readability score reliably

4. Requirements (all are graded)

- **Exactly 3 agents and 3 tasks**, each agent with role, goal, backstory, llm, allow_delegation=False and verbose=True.

- **One built-in tool:** ScrapeWebsiteTool given to the Web Researcher only, created without a fixed URL (the agent passes {url}).
- **One custom tool:** a BaseTool subclass with name, description, args_schema and _run(); given to the LinkedIn Writer only. It must run on its own with `python post_tools.py` (no API key, no internet).
- **Checker rules** (constants at the top of post_tools.py): 150 - 250 words; average sentence at most 20 words; Flesch Reading Ease at least 50; 3 - 5 hashtags. Hashtags are not counted as words.
- **Task wiring:** Task 2 has context=[Task 1]; Task 3 has context=[Task 2, Task 1] and output_file='output/learning_bite.md'.
- **Inputs, not hard-coding:** task descriptions use {url} and {topic}; values are passed only in crew.kickoff(inputs=...).
- **Two topics:** run the crew for the two URLs in the Build Guide; keep both output files (rename the first before the second run).
- **No secrets in code:** OPENAI_API_KEY lives only in .env, which is in .gitignore.

5. What you will be able to do after this

- Choose between a built-in tool and a custom tool, and explain the choice in one sentence.
- Write a custom CrewAI tool with a validated input schema and test it without any LLM.
- Pass work between agents with context, and save a task's result with output_file.
- Read CrewAI's verbose log to prove which agent called which tool with which input.
- Show, with your own runs, how much a model's self-reported word count differs from the real count.

6. What to submit and how it is graded

Item	Marks	What earns full marks
test_scrape.py	10	Checkpoint 1: prints character count, token estimate and the start of the article for both URLs
post_tools.py	25	Checkpoint 2: bad sample FAILs with 4 reasons, your own sample PASSes; all rule values in constants; clear description
crew.py	25	3 agents, 3 tasks, correct tool per agent, context links, output_file, {url}/{topic} inputs, max_iter on the writer
Two output posts	15	Both end with the checker report showing VERDICT: PASS; no fact that is not in the scraped page
notes.md	15	Experiments E1 - E4 with numbers from your runs (word counts, tool calls, token usage)
Code quality	10	Header comment on each file; runs from a clean clone; .env, .venv/ and output/ not committed

Submit: a zip or GitHub link with test_scrape.py, post_tools.py, crew.py, requirements.txt, notes.md, the two output posts and a screenshots/ folder (Checkpoint 2 output, the log lines where each tool is called, and one final post).

7. Stretch goals (optional, no extra marks)

- Rewrite the checker with the `@tool` decorator (as in stock_tool.py). What did you lose compared with BaseTool + args_schema?
- Make Task 2 return structured data with `output_pydantic=` (a Pydantic Outline class). Does Task 3 follow it more closely?
- Add a fourth rule to the checker: no sentence longer than 30 words. Report the longest sentence in the FAIL message.